TestTailor: Generating High-Coverage Tests via Path-Proximal Tests with LLMs

XIAOXUAN ZHOU, Peking University, China and Northeastern University, China
YILING LOU, Fudan University, China
JINHAO DONG, Peking University, China
DAN HAO*, Peking University Shenzhen Graduate School, China and Peking University, China

Automated unit testing is essential for ensuring software quality. Achieving high code coverage through automated unit test generation remains challenging, especially for hard-to-cover branches guarded by complex or deeply nested conditions. Traditional search-based approaches often stagnate at fitness plateaus, while recent LLM-based techniques provide mostly coarse-grained prompts, leaving models to guess how to reach uncovered targets. To address these limitations, we present TestTailor, a neuro-symbolic framework that exploits fine-grained, path-oriented guidance to guide LLM-based test generation. The key idea is to exploit path-proximal tests (i.e., existing test cases whose execution paths closely resemble the target uncovered path) and to analyze their divergence points. By combining this analysis with symbolic constraints (i.e., constraints collected from the target uncovered path using symbolic execution), TestTailor derives actionable path guidance and encodes them into concise prompts that tell the LLM not only what remains uncovered, but also how to reach it. We evaluate TestTailor on the widely used CODAMOSA benchmark comprising 486 Python modules. Results show that TestTailor consistently outperforms state-of-the-art baselines, improving statement coverage by 5.01% and branch coverage by 4.17% on average compared to the best baseline CoverUp, while incurring only about 40% of CoverUp's API cost. Against the hybrid LLM-search-based technique CODAMOSA, TestTailor achieves even larger gains of 12.78% and 13.09% in statement and branch coverage, respectively. Moreover, TestTailor attains the highest coverage accuracy (85.2% vs. 75.3% for CoverUp and 63.8% for TELPA), and demonstrates robustness across different LLM backbones. These results highlight that TestTailor transforms vague coverage goals into precise path-level instructions, enabling LLMs to generate high-coverage test suites more efficiently and accurately.

CCS Concepts: • **Software and its engineering → Software testing and debugging**.

Additional Key Words and Phrases: Automated unit testing, Large language models, Path-proximal analysis

ACM Reference Format:
Xiaoxuan Zhou, Yiling Lou, Jinhao Dong, and Dan Hao. 2026. TestTailor: Generating High-Coverage Tests via Path-Proximal Tests with LLMs. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE013 (July 2026), 22 pages. https://doi.org/10.1145/3797140

*Corresponding author.

Authors' Contact Information: Xiaoxuan Zhou, Key Lab of High Confidence of Software Technologies (Peking University), Ministry of Education, Peking University, Beijing, China and Northeastern University, Shenyang, China, zhouxx@mails.neu.edu.cn; Yiling Lou, Fudan University, Shanghai, China, yilinglou@fudan.edu.cn; Jinhao Dong, Key Lab of High Confidence of Software Technologies (Peking University), Ministry of Education, Peking University, Beijing, China, jhdong@stu.pku.edu.cn; Dan Hao, Peking University Shenzhen Graduate School, Shenzhen, China, Key Lab of High Confidence of Software Technologies (Peking University), Ministry of Education and Peking University, Beijing, China, haodan@pku.edu.cn.

This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2994-970X/2026/7-ARTFSE013
https://doi.org/10.1145/3797140

1 Introduction

Unit testing [27, 30, 41] is a fundamental software quality assurance technique that focuses on exercising individual components in isolation. Beyond verifying correctness, effective unit testing is often measured by its ability to achieve high code coverage, which reflects how thoroughly the codebase has been exercised by tests. However, achieving sufficient coverage requires substantial manual effort in designing diverse and comprehensive test cases, making it a costly and time-consuming process [9, 19]. To address this challenge, automatic test generation [5, 20, 23, 24, 32] has become a central research topic. Among traditional automatic test generation methods, search-based approaches, commonly referred to as Search-Based Software Testing (SBST), are the most widely used, with tools such as Pynguin [23] and EvoSuite [1]. However, these approaches often fall into local optima when faced with complex logic, making it difficult to cover critical execution branches [1]. In a large scale study of SBST on 110 widely used open source projects, Fraser and Arcuri observed that more than 25% of the tested software classes had less than 20% coverage [13].

Recently, with the rapid advancement of large language models (LLMs), a new line of research has emerged that incorporates LLMs into test generation [2, 3, 8, 21, 22, 24, 31, 33, 38, 40]. These methods leverage the comprehension ability of LLMs by conditioning them on contextual information such as source code or natural language specifications, thereby enabling the generation of test cases that are more effective in exercising complex program paths and improving overall coverage.

Despite recent advances, existing approaches, whether search-based or LLM-based, still struggle to generate high-coverage test suites, especially for hard-to-cover branches where synthesizing inputs that satisfy complex and deeply nested conditions is inherently difficult. For search-based techniques, progress relies on mutating existing tests to improve a fitness measure. When inputs must satisfy strict formats or deeply nested constraints, random mutations rarely generate coverage-increasing cases, causing the search to stall at fitness plateaus [21]. LLM-based techniques face a different but related challenge: the guidance provided to the model is often coarse-grained. Some methods attempt to supply as much information as possible—ChatTester [40], for instance, provides the LLM with the complete focal method along with its surrounding class context—including the class name, fields, and the signatures of all methods in the class—while instructing it to generate a corresponding unit test, regardless of the method's size or complexity. TELPA [39] augments the target method with its call chain and counterexample tests, yet merely instructs the LLM to "cover different scenarios or edge cases." As a result, LLMs are often left to guess, generating test inputs that cover only a few simple, easily reachable cases, contribute little to overall coverage, and consume unnecessary computational resources.

Directly synthesizing tests to cover a target path with complex or deeply nested conditions is often challenging for both LLMs and traditional search-based techniques. Our key insight is to leverage existing tests that traverse paths similar to the target path (referred to as *path-proximal tests*). The gap between the path-proximal test and the target path can provide valuable information, which can further help LLMs understand the runtime behaviors of existing tests and reveal why those tests fail to reach the target path. Therefore, such differences can serve as fine-grained hints that guide LLMs in generating new tests specifically aimed at covering the uncovered target paths.

Based on this insight, we propose TestTailor, a neuro-symbolic[1] test generation technique for generating high-coverage tests. For a program under test and its existing test suite, TestTailor aims at generating additional new tests that can cover the uncovered code regions of the program. Given a program and its existing test suite, TestTailor first decomposes the uncovered code into fine-grained target units, so that coverage can be pursued with precision. For each unit, it searches the test suite for a path-proximal test whose execution path is closest to the uncovered one. By

[1]We refer the synergy between LLMs and program analysis as neuro-symbolic in this work.

contrasting this path with the target, TestTailor identifies the first point of divergence. Moreover, TestTailor employs symbolic execution to extract constraints along the target path, referred to as *symbolic constraints*. These constraints—together with the surrounding context—are distilled into a concise prompt that tells the LLM both what branch remains uncovered and how existing tests already behave. In this way, TestTailor transforms the coarse and ambiguous guidance of prior LLM-based methods into precise, path-specific hints. The result is a unit-by-unit generation process that directly targets hard-to-cover branches, reduces redundant test explosion, and fully harnesses the latent information embedded in existing tests.

To evaluate TestTailor, we conduct experiments on CODAMOSA dataset [25], which is a widely adopted benchmark for Python test generation [3, 21, 39] and includes 486 Python modules from 27 widely-used open-source projects. Our experiments show that TestTailor significantly outperforms state-of-the-art test generation techniques across both categories. Compared to the best LLM-based baseline CoverUp, TestTailor improves the statement coverage by 5.01% and branch coverage by 4.17% on average while incurring only about 40% of CoverUp's API cost. Against the hybrid LLM-search-based technique CODAMOSA, TestTailor achieves even larger gains, improving statement coverage by 12.78% and branch coverage by 13.09% on average. In addition, our results also show that TestTailor achieves a coverage accuracy of 85.2%, meaning that the vast majority of its successfully executed tests indeed fulfill their intended coverage targets. In contrast, only 75.3% of CoverUp's and 63.8% of TELPA's generated tests hit their designated targets. Such results confirm that the fine-grained hints in TestTailor enable LLMs to generate tests that more precisely satisfy the intended path coverage goals. Furthermore, we evaluate the generality of TestTailor on different LLM backbones (i.e., `DeepSeek-V3` [11] and `gpt-4o-2024-11-20` [28]), showing the consistent improvements of TestTailor. Lastly, our ablation study demonstrates that each key component contributes to the overall effectiveness of TestTailor.

The main contributions of our study are as follows:

- We propose TestTailor, a novel LLM-based test generation technique that integrates path-proximal test cases and provides fine-grained, unit-level coverage guidance by combining path divergence analysis with symbolic constraints.
- TestTailor overcomes the shortcoming of existing LLM-based test generation by effectively leveraging path-proximal test cases and enabling fine-grained prompt construction, thereby reducing redundancy and achieving greater coverage improvements.
- We conduct a large-scale evaluation on the CODAMOSA benchmark using multiple LLM backends, demonstrating that TestTailor achieves significantly higher statement and branch coverage than state-of-the-art methods while reducing API cost by about 60%.
- We provide an in-depth ablation study, showing that path-proximal guidance is the dominant factor while path constraints provide complementary refinement, offering new insights for future LLM-based test generation research.

2 Motivating Example

To illustrate the challenges faced by existing LLM-based test generation approaches, we consider a concrete example from `thonny.roughparse`. [2] We focus on the function `find_good_parse_start`, where the initial test suite had already covered all preceding statements, leaving only line 200 (highlighted in the left part of Figure 1) and its subsequent sequential statement uncovered. In this setting, both TELPA and CoverUp were applied. TELPA's prompt described the goal only in vague terms ("cover different parts of the function"), whereas CoverUp provided a seemingly

[2] https://github.com/thonny/thonny

more explicit instruction ("line 200 does not execute when tested"). Yet, because line 200 together with its immediate successor were the only uncovered parts, the actual generation target for both approaches was effectively identical: reaching line 200. As shown on the right-hand side of the figure, the black bold text indicates how each approach encodes the current coverage state, while the red bold text specifies the task instruction. Despite their differences in wording, both prompts remain coarse-grained, pointing only to a missing location without offering actionable path-level reasoning.

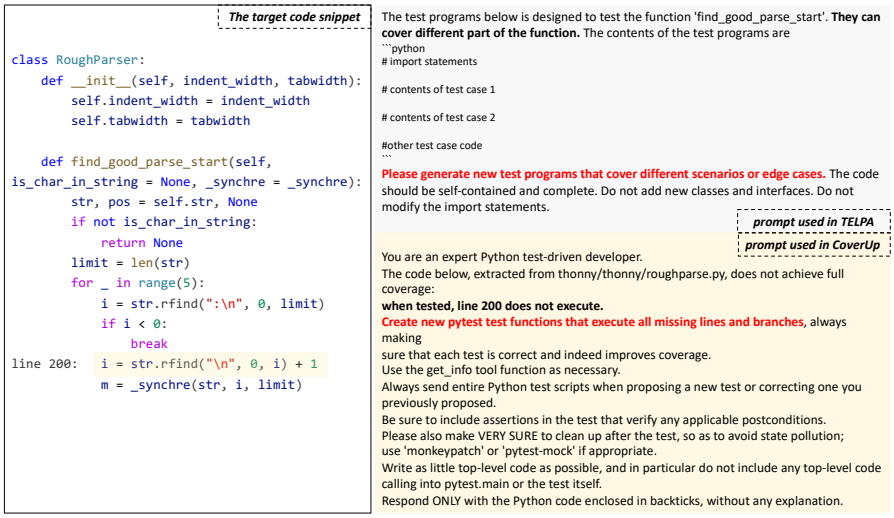

Fig. 1. An example from `thonny.roughparse`: the left side shows the corresponding code snippet, the upper-right box is the prompt used by TELPA, and the lower-right box is the prompt used by CoverUp.

As a result, neither approach was able to generate a test case that executes line 200. Instead, the newly generated tests repeatedly exercised code that had already been covered by the initial suite, failing to contribute any additional coverage. From this example, we draw several conclusions: (i) Coarse-grained prompts that only restate uncovered locations are insufficient to guide the LLM toward hard-to-reach branches. (ii) Even when the uncovered target is unambiguous (e.g., line 200), current approaches still fail without actionable guidance. (iii) Effective test generation requires bridging the gap between knowing what remains uncovered and knowing how to reach it.

These conclusions directly motivate the design of TestTailor, which augments coverage-driven prompting with path-proximal information. By encoding path divergence information and symbolic constraints around uncovered code, TestTailor transforms coverage gaps into concrete guidance, thereby enabling the generation of tests that exercise otherwise unreachable regions.

3 Methodology

We introduce TestTailor, a neuro-symbolic test generation approach that produces executable tests to cover the uncovered code regions of the program under test. The main insight behind TestTailor is to incorporate existing path-proximal tests and critical symbolic constraints along the target path as fine-grained hints. Intuitively, a path-proximal test refers to an existing test case whose execution path is closest to the target uncovered region (see § 3.2.2). On the one hand, the gap between existing path-proximal tests and the target path can help LLMs better understand the runtime behaviors of existing tests and capture why they fail to cover the target path; on the other

hand, the critical symbolic constraints can guide LLMs to focus on the conditions that must be satisfied for reaching the target path. As a result, these hints could precisely guide LLMs to generate tests for specifically uncovered target paths. Figure 2 shows the overall three-stage workflow of TestTailor. For the given program under test P and an existing test suite T, TestTailor generates a new set of tests T' that can increase the coverage of T on P.

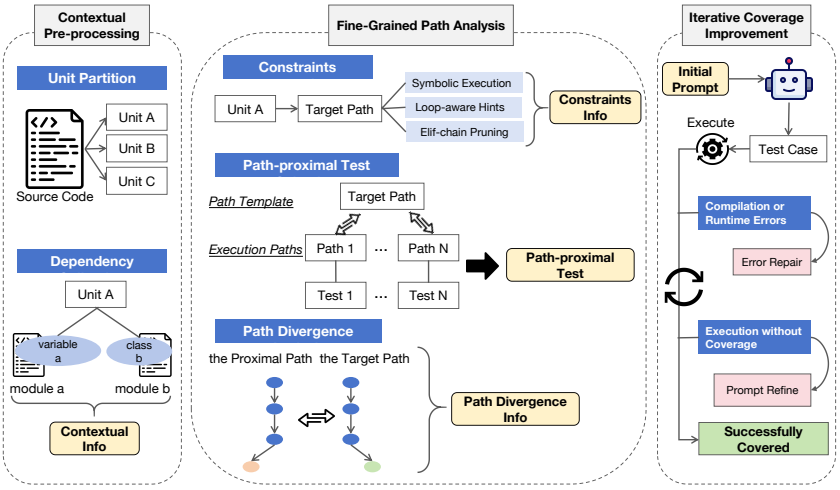

Fig. 2. Overview of the TestTailor workflow.

Contextual Pre-processing. In the first stage, TestTailor first partitions the uncovered code into *smallest contiguous execution units* and then collects the dependencies for all relevant program elements (e.g., functions, classes, or variables) involved in each uncovered unit. Such a fine-grained partition and the dependency collection aim at preparing both succinct and complete code contexts for LLMs during the subsequent test generation/improvement process.

Fine-Grained Path Analysis. In the second stage, TestTailor first applies symbolic execution to extract the path constraints, then analyzes the execution paths of existing test cases to identify path-proximal tests, and finally compares these paths with the target path to locate path divergence.

As shown in Figure 5, it is a realistic case from `sanic.config`. [3] The target unit is highlighted in dark gray and labeled as "The uncovered unit". Starting from this unit, TestTailor recursively traces upward along the black arrows on the left until it reaches the enclosing function node, and then reverses the chain to construct the target path, illustrated by the red dashed arrow on the right side of the figure. From this path, the required constraints are extracted, as displayed in the box in the figure. Next, the target path is compared with the execution paths of existing test cases to identify the most path-proximal test. Once this test is found, TestTailor compares its actual execution path with the target path to locate the first divergence point, highlighted in the dashed box on the right. By consolidating these elements into a concise prompt, TestTailor guides the LLM to focus on the uncovered behavior while grounding generation in realistic usage patterns.

Iterative Coverage Improvement. In the third stage, TestTailor guides LLMs to generate new tests based on the selected test and the collected symbolic constraints. If the generated test fails to increase the coverage, TestTailor iteratively leverages LLMs to fix it based on the symbolic constraints. To control computational cost and avoid diminishing returns, each unit is allotted at most three iterations. Once all uncovered units are processed, the workflow terminates.

[3] https://github.com/sanic-org/sanic

3.1 Contextual Pre-Processing

Overlong contexts may overwhelm LLMs, whereas incomplete contexts can degrade their effectiveness. To provide LLMs with contexts that are both *succinct* and *complete*, TestTailor begins the contextual pre-processing stage by partitioning the uncovered code into the smallest contiguous execution units (i.e., *Unit Partition*), which ensures fine-grained and concise context construction. It then gathers the dependencies of all functions and classes within each unit (i.e., *Dependency Collection*), thereby enabling the construction of complete contexts.

Unit Partition. After executing all existing tests T on the program under test P, TestTailor identifies the set of uncovered code B. It then partitions B into fine-grained units, denoted as $B = \{b_i\}$. Unlike traditional basic blocks [17], which are restricted to linear sequences of statements, a *unit* in TestTailor represents a *smallest contiguous execution unit* that can be either a linear sequence or a minimal branching construct without further nested conditionals (e.g., a simple `if` statement), as illustrated in Figure 3. This design choice is intentional: such granularity ensures that each unit remains a semantically self-contained logic chunk, which is more effective for LLMs to understand and satisfy than fragmented basic blocks.

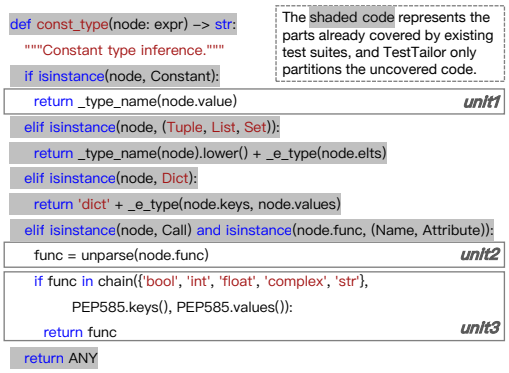

Fig. 3. A practical case of dividing uncovered code into code units in TestTailor on a snippet from `apimd.parser`.

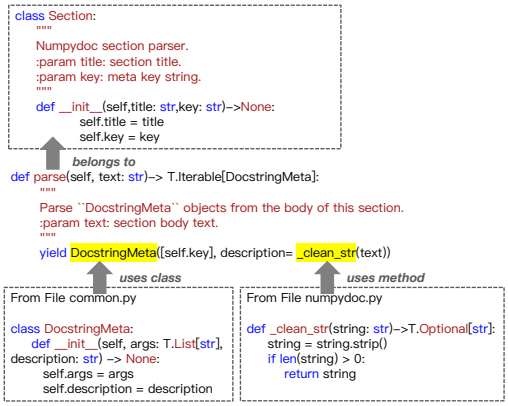

Fig. 4. A practical case of gathering relevant code context information in TestTailor on a snippet from `docstring_parser.numpydoc`.

Dependency Collection. Only providing the target unit to the LLM is far from sufficient, as the unit often relies on variables, functions, or classes defined in other parts of the project (e.g., in different source files). Therefore, to prepare a complete input context, TestTailor further collects the dependencies for each target unit to enrich the input context. In particular, TestTailor performs static analysis over the entire codebase of the program under test, constructing abstract syntax trees (ASTs) and class dependency graphs to accurately resolve references across files and modules. On top of this global analysis, TestTailor applies backward program slicing starting from all statements in the target unit. This allows it to identify the other functions, classes, and variables that the target unit depends on, and extract their corresponding definitions. Moreover, whenever a class is involved, TestTailor also includes the class definition together with its `init` method and relevant docstrings. As a result, unrelated code is pruned while only semantically relevant dependencies are preserved, enabling the construction of a complete yet concise input context for the LLM. Figure 4 illustrates this process with the `parse` function from the `docstring_parser`[4]

[4]https://github.com/rr-/docstring_parser

project. The `parse` function belongs to the `Section` class; accordingly, TestTailor extracts the class definition together with its `init` method and relevant docstrings. Moreover, since `parse` depends on the `DocstringMeta` class from the `common` module and the `_clean_str` method from the `numpydoc` module, their corresponding information is also incorporated into the prompt.

3.2 Fine-Grained Path Analysis

In the second stage, TestTailor conducts fine-grained path analysis to derive comprehensive information that can guide LLMs in generating tests for covering the specific unit. In particular, TestTailor first collects a full set of symbolic constraints that must be satisfied along the target path (§ 3.2.1). While directly generating a test satisfying all the symbolic constraints can be too challenging for LLMs, we further identify the path-proximal test from existing tests, which has the closest execution path to the uncovered target path (§ 3.2.2). The path-proximal test could serve as a reference example for LLMs to generate tests with similar execution path to the target path. Moreover, we further analyze the gap between the selected path-proximal test and the target path by localizing the first divergence point along the target path (§ 3.2.3). Providing such a path gap to LLMs can further help models understand why the existing path-proximal test cannot cover the target path. In summary, all the collected information (i.e., symbolic constraints, path-proximal test, and divergence point) could together guide LLMs towards generating tests specifically for the uncovered unit.

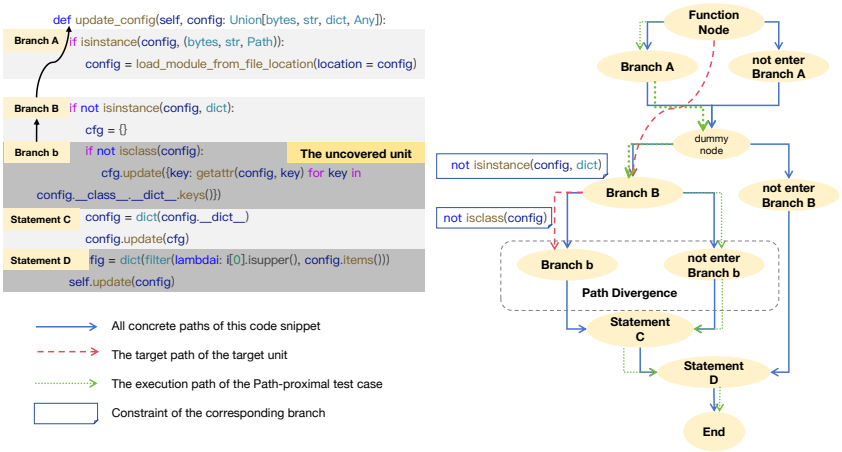

Fig. 5. An illustration of constructing the target path, constraints extraction, and path divergence analysis. The left side shows the original function `update_config` from `sanic.config`, where the uncovered unit (Branch b) is highlighted in dark gray and marked as "The uncovered unit." The right side depicts the corresponding control-flow structure, where the red dashed line represents the extracted target path and the green dotted line denotes the execution path of a path-proximal test case. Note that the dummy node in the figure has no semantic meaning; it is introduced solely to simplify the visualization.

3.2.1 Constraints Extraction via Symbolic Execution. For an uncovered unit b_i, we define the path intended to cover it as our target path p_t. Crucially, p_t does not correspond to a concrete execution trace; instead, it is an entry-to-target skeleton constructed as a path template. Specifically, we traverse the program's AST upward from the uncovered unit to its function root (typically a function or class definition); reversing this chain yields a structural template that spans from the function entry to the target unit. This template preserves the full prefix but abstracts away specific,

non-essential branching choices. For instance, if multiple sibling branches eventually converge to the target, the template remains agnostic to the specific choice between them, focusing only on mandatory structural checkpoints.

Figure 5 illustrates this process with an example from `sanic.config`. The left part shows the original function code containing the uncovered unit (the dark gray part marked with "The uncovered unit"), while the right part depicts the corresponding control-flow structure. Following the black arrows on the left, TestTailor recursively traces upward from the uncovered unit to the function node, and then reverses the chain to form a top-down path template. The red dashed line in the right-hand figure marks the extracted target path. In contrast, the green dotted line shows the execution path of a path-proximal test case. Unlike this concrete execution trace—which may reach Branch B either by entering Branch A or by skipping it—the target path excludes sibling alternatives. This distinction underscores that the target path is not a runtime trace but a structural template that can avoid combinatorial path explosion.

TestTailor extracts precise constraints that must be satisfied to traverse the target path. Specifically, TestTailor leverages symbolic execution to collect all branch conditions along the path.

In addition to harvesting the Boolean predicates that guard ordinary branches, TestTailor performs dedicated handling of the following two patterns: (1) Loop-aware hints: for `while`/`for` headers TestTailor records both the guard predicate and an iteration-count or entry-rank hint that tells the LLM how many times the loop must execute to reach the uncovered statement. (2) Elif-chain pruning: for consecutive `if-elif` arms TestTailor negates every earlier guard, yielding a constraint that uniquely selects the intended arm while excluding all its predecessors. Consider again the case in Figure 5. Along the extracted target path, there are two conditional statements, Branch B and Branch b. Consequently, the corresponding path constraints are

```
not isinstance(config, dict) and not isclass(config).
```

3.2.2 Path-proximal Test Selection. A path-proximal test is the existing test case whose execution path most closely resembles the target path p_t of an uncovered unit b_i. Intuitively, it is the test that follows p_t as far as possible before taking a different branch. To capture this formally, let $\mathcal{P}$ be the set of execution paths induced by the existing test suite. The path-proximal test is defined as

$$p_r^* = \arg\max_{p_r \in \mathcal{P}} \texttt{Jaccard}(p_r, p_t),$$

with ties broken by choosing the path that shares the longest common prefix with p_t up to the first divergence toward b_i. The rationale behind using path similarity as a criterion for test case selection is rooted in the observation that existing test suites often contain tests that, while not directly covering the target path, traverse adjacent execution paths. These tests typically satisfy most of the same path conditions as the target path, differing only in a small subset of conditions. As a result, they inherently capture relevant control flow structures and variable interactions that are crucial for understanding the execution context of the target code. By leveraging such similarities, TestTailor reduces the complexity and improves the precision of generating new test cases.

To operationalize this insight, TestTailor first converts the program under test into a control-flow graph (CFG) derived from its AST and organizes the CFG as a dominator tree for efficient reachability queries. Each uncovered unit is mapped to a target CFG node. Starting from this node, TestTailor performs a hierarchical search over the dominator tree:

- **Sibling-level search.** The algorithm looks for existing tests that execute any sibling of the target path. If multiple tests are found, it ranks them by Jaccard similarity between their runtime paths and the target path, preferring the test with the highest overlap and shortest suffix.

- **Ancestor-level fallback.** If no suitable test exists among siblings, the search ascends to the parent node and repeats the Jaccard-based ranking, continuing until a test is located or the root of the tree is reached.

Figure 5 illustrates this process. The red dashed line denotes the target path leading to the uncovered unit. In the sibling-level search, TestTailor first looks for tests that enter Branch B but skip Branch b. Such a test is indeed found, shown by the green dotted line in the figure. If no such candidate were available, the algorithm would fall back to the ancestor level, moving upward to the parent of Branch b and repeating the search. If multiple tests are found, it ranks them by Jaccard similarity between their runtime paths and the target path, preferring the test with the highest overlap; if ties occur, TestTailor further selects the test whose remaining suffix beyond the common prefix is shortest, as this indicates the closest divergence from the target path. This ascending strategy ensures that a highly path-proximal test is selected, providing TestTailor with a compact yet rich execution template for the subsequent constraint-guided generation phase. Algorithm 1 details the complete procedure.

Algorithm 1 Hierarchical Search for the Most Path-Proximal Test Case

Input: B_{target}: the target uncovered code unit
S_{tests}: the existing test suite
G_{CFG}: the Control Flow Graph of the program
T_{Dom}: the Dominator Tree of the program
Output: T_{seed}: the most path-proximal test case, or `null`

```
function FindSeedTest(B_target, S_tests, G_CFG, T_Dom)
    Scope ← GetSiblingUnits(B_target, T_Dom)                ▷ Start search at Level 0
    while Scope ≠ null do
        C_tests ← FindCoveringTests(Scope, S_tests)
        if C_tests ≠ ∅ then                                 ▷ Candidates found at the current level
            T_seed ← null
            maxSim ← −1
            minSuffix ← ∞
            for each T_cand ∈ C_tests do
                sim ← CalculatePathSimilarity(T_cand, B_target, G_CFG)
                suf ← SuffixLength(T_cand, B_target, G_CFG)
                if sim > maxSim or (sim = maxSim and suf < minSuffix) then
                    maxSim ← sim
                    minSuffix ← suf
                    T_seed ← T_cand
                end if
            end for
            return T_seed                                   ▷ Return the best candidate
        end if
        Scope ← GetParentScope(Scope, T_Dom)                ▷ Expand to the parent level
    end while
    return null                                             ▷ No similar test found
end function
```

3.2.3 Path Divergence Analysis. While path-proximal tests can be helpful for LLMs to generate tests with similar paths close to the target path, it can still be challenging for the generated tests to

cover the uncovered unit. To further refine the guidance for LLMs in test generation, TestTailor performs a targeted contrastive analysis between the execution path of the selected reference test and the intended target path. TestTailor employs dynamic execution tracing to meticulously analyze the execution flow of the reference test case in relation to the target path. This process involves monitoring the program's runtime behavior to pinpoint the divergence point—specific juncture where the execution path of the reference test case deviates from the intended target path.

Consider again the example in Figure 5. Both the red dashed line (the target path) and the green dotted line (the actual execution path of the reference test) pass through Branch B. However, the target path enters Branch b, whereas the reference test does not, and the divergence occurs precisely at this point. Mapping this back to the source code yields the following analysis:

The existing test case diverges at:

```
if not isclass(config):
```

Target path proceeds to:

```
cfg.update({key: getattr(config, key) for key in config.__class__.__dict__.keys()})
```

Existing test path proceeds to:

```
config.__class__.__dict__.keys()
```

3.3 The Iterative Framework

Once a context-rich prompt has been assembled (§3.1–§3.2), TestTailor enters the third stage: an iterative generate-and-refine loop whose sole objective is to turn the prompt into an executable test that reaches the uncovered branch. The prompt template (Figure 6) concatenates (1) the target code unit and its contextual information extracted in §3.1, (2) the path-proximal reference test case and contrastive analysis from §3.2, and (3) explicit formatting instructions for test generation. This structured prompt is submitted to the LLM, whose output is parsed into an executable candidate test. Each candidate test then undergoes compilation and execution. If the generated test successfully covers the target unit, the unit is marked as resolved and no further refinement is needed. Otherwise, the workflow distinguishes between two failure modes:

- **Compilation or runtime errors.** In this case, TestTailor extracts the error message and incorporates it into a repair prompt, which is appended to the original context. The LLM is then invoked again to generate a corrected version of the test.
- **Execution without coverage.** If the test runs correctly but fails to exercise the target unit, TestTailor first evaluates whether the generated test case is still of potential utility. Specifically, if the test achieves coverage of the function containing the target unit (but misses the target unit itself), the workflow treats this test as a new "path-proximal" seed. By reanalyzing its execution path against the intended target path, TestTailor refines the prompt to highlight the remaining divergence and requests a more targeted test from the LLM. To maintain prompt conciseness and avoid exceeding the context window, TestTailor replaces the previous proximal test with this newly generated, "closer" test case rather than appending it. Otherwise, if the test fails to even reach the enclosing function, the workflow discards it as uninformative and continues to use the original reference test for regeneration. This design ensures that any partial progress toward the target path is exploited rather than wasted, enabling the workflow to iteratively narrow the gap to the uncovered unit while avoiding misguidance from irrelevant executions.

Each uncovered unit is allowed at most three iterations. This threshold is established based on prior research [22] and our own pilot study, which indicated that success rates typically plateau after the third attempt. Specifically, we found that a fourth iteration yields less than 1% additional coverage while significantly increasing API costs. When every unit has either succeeded or exhausted its

budget, the workflow terminates. The union of all generated tests together with the initial test suites constitutes TestTailor's final test suite.

Function under Test
<the target function>

Relevant Information from the entire project
<relevant information such as methods and classes..>

The Target code unit you need to cover
<the target code unit>

Contextual Preprocessing

To reach the target code, the test case needs to satisfy conditions along the path:
<symbolic constraints>

The most Path-Proximal test case
<the contents of the most path-proximal test case>

Path Divergence Analysis
The existing test case diverges from the target path at: Line A
Target path continues to: Line B
But existing test path continues to: Line C

Fine-Grained Path Analysis

Your task
Create a test case that targets the branch containing our target code.
Use the reference test case as guidance and complete the function body only.
** Important: Only provide the completed test case starting from "class Test" line. Do not include import statements. **

```Python
# necessary import statements
class Test(unittest.TestCase):
    @timeout_decorator.timeout(1)
    def test_case_XX(self):
        """complete the test case here"""
```

Explicit Formatting Instructions for Test Generation

Fig. 6. The template of TestTailor in the initial generation stage.

4 Experimental Setup

To evaluate the performance of TestTailor, we formulate the following research questions (RQs).

- **RQ1:** How does TestTailor compare with state-of-the-art LLM-based test generation techniques in terms of effectiveness?
- **RQ2:** How do path-analysis components, including path constraints, path-proximal tests, and path divergence analysis, contribute to the effectiveness of TestTailor?
- **RQ3:** How generalizable is TestTailor when built upon different LLMs?
- **RQ4:** How cost-effective is TestTailor?

4.1 Implementation of TestTailor

We implemented TestTailor as a lightweight, fully-automated Python tool-chain. The pipeline begins with the standard `ast` module to parse the program under test into an abstract syntax tree (AST). From this AST we construct an in-memory, tree-shaped IR whose nodes correspond to individual statements or minimal blocks; each node is annotated with its syntactic type (*if*, *for*, *def*, etc.), indentation level, and parent–child relationships, enabling fine-grained slicing and navigation. On top of this representation, TestTailor applies static reference resolution: by analyzing variables and function calls in the AST and its control-flow graph, TestTailor traces each usage back to its definition. When definitions reside in other modules, TestTailor follows `import` statements to recursively analyze the corresponding ASTs, ensuring that slices include complete function bodies, class implementations, and initializations across files. Dynamic execution traces are captured via Python's built-in `trace` facility. We apply a filtering layer that retains only events within the current module, yielding compact yet complete runtime paths. Coverage measurement relies on the mature

`coverage` package, which provides line- and branch- metrics without additional instrumentation overhead.

4.2 Evaluation Setup

4.2.1 Dataset. In the evaluation, we use the dataset released by Lemieux et al. [21], which was widely used to evaluate test generation techniques [3, 21, 39]. This dataset contains 486 modules sampled from 27 open-source python projects. The dataset was curated to represent realistic and diverse testing scenarios, encompassing a variety of program structures, libraries, and application domains.

4.2.2 Baselines for RQ1 and RQ3. To answer RQ1 and RQ3, we compare TestTailor against three state-of-the-art LLM-based test generation methods: **CODAMOSA**, **CoverUp**, and **TELPA**.

CODAMOSA [21] integrates an LLM into Pynguin [23], an automated unit test generation framework for Python, by switching from search to model-generated seed tests whenever coverage stalls, then resumes search to escape local optima and boost coverage. CoverUp [3] guides an LLM to iteratively generate high-coverage Python regression tests by leveraging coverage-guided prompts, a `get_info` tool function for on-demand context, and continued dialogue refinement until targeted lines and branches are exercised. TELPA [39] combines object construction analysis and branch dependency analysis with coverage-guided counterexample sampling to craft LLM prompts, enabling the generation of tests for hard-to-reach branches involving complex objects and interprocedural dependencies.

We select these methods as baselines because they represent the current state of the art in LLM-based coverage-oriented test generation strategies. Other contemporaneous approaches (e.g., SymPrompt [31]) were not included due to the absence of reproducible open-source implementations at the time of evaluation, which prevents fair and controlled comparison.

For a fair comparison, CODAMOSA first runs for two minutes to harvest "easy" branches, following previous work's setup. [5] Since the original CODAMOSA implementation was tailored to Codex [26], we modified its configuration to support the LLMs used in our experiments. This set is then used as the seed input for CoverUp, TELPA, and TestTailor. For each method, we set a maximum runtime of 20 minutes per module, as our pilot runs showed that even modules with a large number of statements complete within this limit. This threshold thus ensures that all techniques have sufficient opportunity to demonstrate their capabilities while avoiding resource waste in rare cases with unusually long executions. In addition, CODAMOSA is also executed for 20 minutes to serve as a direct comparison. We use `gpt-4o-2024-11-20` as the default setting, but switch to `deepseek-v3` in RQ3, with the `temperature` consistently set to 1 across all experiments.

4.2.3 Variants for RQ2. To answer RQ2, we design three variants of TestTailor to isolate the contribution of its path-related components in the pipeline shown in Figure 6:

- **TestTailor$_{\text{Basic}}$**: Retains only the Contextual Pre-processing stage, providing the target method and its surrounding contextual information (§ 3.1), while removing all components in Fine-Grained Path Analysis.
- **TestTailor$_{\text{Constr}}$**: Keeps Contextual Pre-processing and the symbolic path constraints derived in Fine-Grained Path Analysis, but removes both path-proximal test case retrieval and branch divergence analysis.
- **TestTailor$_{\text{PathSim}}$**: Keeps Contextual Pre-processing, path-proximal test case retrieval, and branch divergence analysis, but removes the symbolic path constraints.

[5] Pilot tests on a small benchmark showed the vast majority of low-hanging branches are covered within this window, so the short warm-up curbs redundant LLM calls and keeps testing cost-effective [39].

4.2.4 Metrics. We evaluate the effectiveness of all methods primarily using *average statement coverage* and *branch coverage*. Beyond these, we also introduce the metric of *effective coverage hit rate*, defined as the ratio between the number of generated test cases that achieve the intended coverage targets and the number of test cases that execute successfully. This metric captures how precisely each method steers test generation toward exercising the designated paths. For RQ4, we further record computational overhead, the number of LLM invocations, the success rate of generating executable test cases, and the coverage improvement curves over LLM calls. Note that CODAMOSA is a hybrid approach: its coverage gains primarily come from the underlying SBST engine, with the LLM used only sporadically as a seed provider. Therefore, the metrics described above are not applied to CODAMOSA.

5 Results and Analysis

5.1 RQ1: Effectiveness of TestTailor

5.1.1 In terms of Coverage metrics. Table 1 reports the statement and branch coverage achieved by CODAMOSA, CoverUp, TELPA, and TestTailor across all 27 projects in the CODAMOSA dataset. The column "CODAMOSA-2min" in Table 1 shows the coverage achieved by running CODAMOSA for two minutes to generate the initial test suite. As discussed in § 4.2.2, the results of CODAMOSA-20min, CoverUp, TELPA, and TestTailor are all built upon this CODAMOSA-2min baseline. From an overall perspective, TestTailor achieves the best performance in both statement and branch coverage. On average, it reaches 80.87% statement coverage and 74.51% branch coverage, ranking consistently higher than all baselines. When examining the number of projects where each method attains the highest coverage on both metrics simultaneously, TestTailor dominates with 16 projects, followed by CoverUp and CODAMOSA-20min with 4, TELPA with 3. This clearly demonstrates TestTailor's superior effectiveness across a broad range of projects.

To statistically validate the effectiveness of TestTailor, we conducted repeated-measures ANOVA[6] [29] and pairwise paired t-tests [34], the results are shown in Table 2. The repeated-measures ANOVA indicates significant differences among all methods for both statement and branch coverage ($p < 0.001$). Pairwise paired t-tests show that TestTailor significantly outperforms each baseline ($p < 0.001$), including CoverUp, the strongest competing method. This demonstrates that TestTailor's improvements are both consistent and statistically robust.

To better understand TestTailor's effectiveness, we categorize projects based on their initial branch coverage from CODAMOSA-2min, which also reflects testing difficulty: difficult projects have low initial coverage (below 40%), while simple projects have high coverage (above 80%). In difficult projects such as `ansible`, `tqdm` and `dataclasses_json`, TestTailor consistently outperforms other methods. For example, in `tqdm`, branch coverage rises from 6.43% (CODAMOSA-2min) to 44.74% with TestTailor, compared with 9.59% for CODAMOSA-20min, 26.98% for CoverUp, and 32.33% for TELPA. Similarly, `dataclasses_json` increases from 23.13% to 60.85% with TestTailor, while CODAMOSA-20min achieves 23.84%, CoverUp 48.98%, and TELPA 34.07%. These results highlight TestTailor's ability to explore hard-to-reach branches and handle complex, rarely executed paths. In contrast, for simple projects such as `python-string-utils` and `sty`, where baseline coverage is already high, TestTailor achieves performance comparable to the strongest baselines without dramatic gains (e.g., 100% for CODAMOSA-20min versus 98.89% for TestTailor, 99.45% for CoverUp, and 97.00% for TELPA). Overall, this pattern shows that TestTailor excels in

[6]Repeated-measures ANOVA is a statistical test that extends standard ANOVA to cases where the same subjects (here, software projects) are measured under multiple methods. It accounts for the fact that results from the same subject are correlated, thus reducing the influence of variability between subjects.

Table 1. Comparison of CODAMOSA, CoverUp, Telpa and TestTailor in terms of statement coverage and branch coverage. All values are expressed as percentages.

Package	CODAMOSA 2min		CODAMOSA 20min		CoverUp		TELPA		TestTailor	
	Stmt Cov	Branch Cov	Stmt Cov	Branch Cov	Stmt Cov	Branch Cov	Stmt Cov	Branch Cov	Stmt Cov	Branch Cov
ansible	25.89	29.65	32.92	38.10	41.30	43.40	41.49	47.17	**47.80**	**52.21**
apimd	78.92	74.83	85.49	82.12	83.17	80.13	83.17	80.13	**89.75**	**86.42**
blib2to3	40.57	35.63	53.64	52.57	69.49	61.86	68.25	57.04	**73.73**	**66.52**
codetiming	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**
cookiecutter	59.48	21.30	64.22	26.90	80.60	54.30	89.22	52.80	**92.67**	**61.90**
dataclasses_json	33.10	23.13	33.21	23.84	52.83	48.98	40.88	34.07	**65.92**	**60.85**
docstring_parser	87.11	76.92	92.53	86.81	**97.94**	**96.70**	92.27	91.76	96.65	95.05
flutes	86.79	84.17	89.81	88.33	**95.09**	**95.83**	91.32	89.17	94.72	93.33
flutils	62.20	65.06	70.91	70.74	82.44	80.11	66.49	68.47	**84.45**	**85.51**
httpie	46.64	61.55	41.24	53.29	**80.04**	75.72	65.02	68.47	78.92	**85.38**
isort	91.33	90.00	91.33	90.00	91.33	90.00	**98.67**	**100.00**	96.67	95.00
mimesis	91.99	88.38	92.02	88.68	89.92	85.30	**94.26**	**91.61**	93.88	89.62
py_backwards	63.27	70.57	63.27	70.57	75.89	80.82	65.33	53.98	**83.78**	**80.89**
pyMonet	87.56	75.93	88.91	77.78	96.61	89.51	95.02	**90.12**	**97.74**	89.51
pypara	62.51	29.38	**84.67**	**68.25**	81.44	67.77	76.92	61.61	80.70	67.30
pySnooper	39.10	31.46	29.66	25.08	66.51	62.64	46.67	40.03	**72.77**	**65.54**
python-string-utils	97.69	96.98	**100.00**	**100.00**	99.83	99.45	97.70	97.00	**99.83**	98.89
pytutils	66.49	31.30	67.89	34.50	72.28	50.95	72.28	**53.56**	**79.82**	**53.56**
sanic	65.50	56.99	66.08	60.16	65.50	56.99	71.33	62.53	**73.89**	**65.70**
semantic_release	53.72	37.80	67.03	46.65	73.77	58.71	67.75	43.07	**81.87**	**62.00**
sty	94.95	94.00	97.98	96.00	**100.00**	**100.00**	96.97	98.00	97.98	98.00
thefuck	56.19	42.16	56.53	42.91	75.33	64.93	63.94	49.25	**82.63**	**71.64**
thonny	31.84	18.10	54.12	43.10	66.85	59.91	68.16	60.78	**71.72**	**62.50**
tornado	56.84	50.33	**66.99**	61.69	58.43	52.68	57.85	52.21	65.98	**61.78**
tqdm	21.30	6.43	26.04	9.59	32.50	26.98	37.90	32.33	**51.79**	**44.74**
typesystem	76.69	72.75	**90.00**	**88.40**	85.70	82.92	75.92	71.19	84.93	82.40
youtube_dl	31.17	30.42	31.86	32.37	33.47	32.51	33.35	32.75	**42.94**	**35.45**
Average	63.29	55.38	68.09	61.42	75.86	70.34	72.52	65.89	**80.87**	**74.51**

Table 2. Statistical analysis results: Repeated Measures ANOVA and paired *t*-tests comparing TestTailor with baseline methods.

Method / Comparison	Statement Coverage		Branch Coverage	
	F / t value	p value	F / t value	p value
Repeated Measures ANOVA (All Methods)	26.904	2.49×10^{-15}	25.024	1.58×10^{-14}
TestTailor vs CODAMOSA 20min	5.115	2.48×10^{-5}	5.029	3.11×10^{-5}
TestTailor vs Coverup	5.051	2.94×10^{-5}	4.479	1.33×10^{-4}
TestTailor vs TELPA	5.661	5.94×10^{-6}	4.815	5.48×10^{-5}

challenging projects with low initial coverage, while reliably maintaining strong performance in easier projects.

5.1.2 In terms of Coverage Accuracy. In addition to overall coverage, we further compare the coverage accuracy of the generated tests, i.e., how precisely each technique retains tests that truly achieve the intended coverage. Although all three approaches (CoverUp, TELPA, and TestTailor) adopt a filtering step after executing the generated tests, their retention criteria differ in strictness. Specifically, CoverUp queries the LLM with code fragments consisting of several lines, and a test is preserved if it introduces any new coverage on the corresponding fragment [3]. TELPA, on the other hand, constructs prompts around entire methods and retains a test if it covers any previously

uncovered part of that method [39]. In contrast, TestTailor formulates prompts at the granularity of the smallest contiguous execution unit, and only keeps a test when the unit is fully covered.

Table 3. Coverage accuracy of CoverUp, TELPA and TestTailor, measured as the ratio of tests that achieve their intended coverage targets over all successfully executed tests.

Metric	TELPA	CoverUp	TestTailor
Coverage Accuracy (%)	63.8	75.3	**85.2**

To quantify coverage accuracy, we compute the ratio between the number of tests that fulfill their coverage target and the total number of tests that execute successfully. As shown in Table 3, TestTailor achieves the highest accuracy of 85.2%, compared with 75.3% for CoverUp and 63.8% for TELPA. This indicates that despite CoverUp and TELPA adopting looser acceptance criteria, they are less effective at generating tests that meet their intended coverage goals. By leveraging fine-grained prompts and precise execution-unit targets, TestTailor delivers more accurate coverage while maintaining a competitive success rate.

Conclusion 1: Effectiveness of TestTailor

TestTailor consistently outperforms baselines in both statement and branch coverage, with the most significant gains on difficult projects with low initial coverage. It maintains competitive performance on simpler projects with already high baseline coverage. Additionally, TestTailor achieves the highest coverage accuracy, producing tests that more reliably fulfill intended goals.

5.2 RQ2: Impact of Path-Oriented Components on TestTailor's Effectiveness

To investigate the contribution of path-oriented components, we designed three ablated variants of TestTailor as illustrated in § 4.2.3. Figure 7 quantifies the individual contribution of each path-oriented component by comparing the full TestTailor configuration against three ablated variants.

When disabling Path-Proximal Test + Divergence Analysis ($\text{TestTailor}_{\text{Constr}}$), coverage drops to 72.8% statement and 66.8% branch—decreases of 8.1% and 7.7% relative to the full system. This loss is almost three times larger than the impact of removing path constraints alone, showing that identifying the closest existing execution and locating the first divergence point is a major driver of effectiveness. In contrast, disabling path constraints ($\text{TestTailor}_{\text{PathSim}}$) causes smaller but still measurable reductions of 2.7% for statements and 2.3% for branches, suggesting that once the neighbourhood is fixed, fine-grained path constraints still provide valuable guidance for steering the solver toward uncovered branches. Finally, when both components are removed ($\text{TestTailor}_{\text{Basic}}$), coverage declines by 12.4% for statements and 12.1% for branches—roughly the sum of the individual penalties—indicating that the two mechanisms complement each other: path-proximal guidance helps steer the search toward promising regions, while path constraints prune these regions with precise conditions.

Without steering, constraints are applied to an unfocused space and yield limited gains; without constraints, steering alone cannot fully exploit the discovered neighbourhood. Together, the two mechanisms explain the superior coverage reported in RQ1.

Conclusion 2: Impact of Path-Oriented Components in TestTailor

Both path-proximal guidance and path constraints contribute significantly to TestTailor's effectiveness. The former is the dominant driver, while the latter provides refinement. Their integration yields the best performance, as observed in RQ1.

5.3 RQ3: Generalizability of TestTailor

To verify whether the gains observed in RQ1 are tied to the specific LLM (`GPT-4o`), we repeated the entire pipeline while swapping the backend for `DeepSeek-V3`. Figure 8 summarises the resulting branch-coverage averages across the 27 projects. The ranking of all approaches is preserved regardless of the underlying model: CODAMOSA-20min < TELPA < CoverUp < TestTailor. The absolute coverage values exhibit a reduction in the range of 0 to 2% when transitioning from `GPT-4o` to `DeepSeek-V3`, while TestTailor consistently achieves the largest lead, demonstrating that TestTailor is not contingent on a single proprietary model.

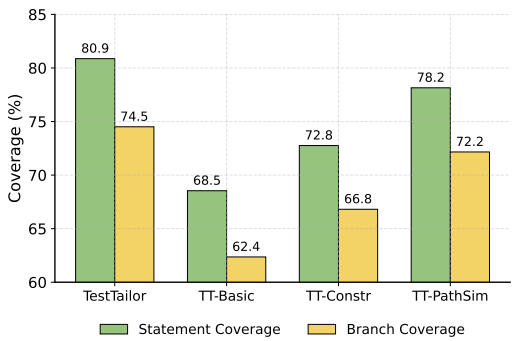

Fig. 7. Average statement and branch coverage of TestTailor and its ablated variants. The results show that the full TestTailor consistently outperforms its simplified versions, highlighting the contribution of each component.

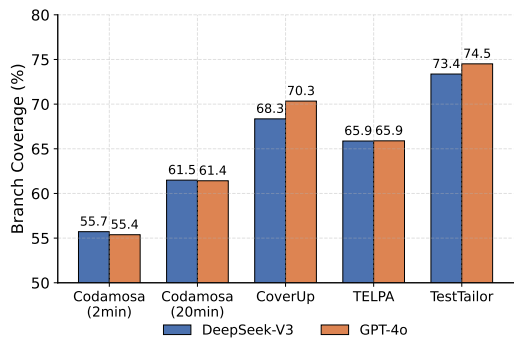

Fig. 8. Average branch coverage comparison between `DeepSeek-V3` and `GPT-4o`. The results show that TestTailor is not tied to a specific LLM.

Conclusion 3: Generalizability of TestTailor

The effectiveness of TestTailor is not tied to a specific LLM.

5.4 RQ4: Cost-Effectiveness of TestTailor

Table 4 compares CoverUp, TELPA, and TestTailor in terms of both overall cost and the proportion of LLM calls that successfully improve coverage. TELPA incurs the lowest monetary cost ($14.34) but also suffers from the lowest success rate (0.14), indicating that the majority of its calls do not yield meaningful coverage improvements. In contrast, CoverUp and TestTailor exhibit comparable success rates (0.29 and 0.31, respectively), but TestTailor achieves this with significantly lower cost than CoverUp ($19.94 vs. $51.90). This demonstrates that TestTailor not only generates more effective test cases per LLM call, but also offers a substantially better trade-off between coverage gains and resource consumption, achieving strong effectiveness at a moderate expense.

Table 4. Comparison of CoverUp, TELPA, and TestTailor in terms of LLM call effectiveness and overall cost.

Metric	CoverUp	TELPA	TestTailor
Overall Cost ($)	51.90	14.34	19.94
LLM Calls Successful Rate	0.29	0.14	0.31

Figure 9 further illustrates the coverage growth as a function of LLM call ranges on the `ansible`[7] package. We choose `ansible` as a representative case because it is the largest project in the dataset, comprising 237 modules—nearly half of the total—and thus highly indicative of the overall trend. For fairness, we only count LLM calls that directly contribute to test generation; auxiliary calls unique to TELPA (e.g., querying method functionality) [39] and CoverUp (e.g., tool invocation) [3] are excluded. Each curve's endpoint (denoted by a circle) corresponds to the final coverage achieved by the respective technique, while the extended line beyond the circle represents additional experiments conducted to verify whether the performance had been fully exhibited. Notably, the curves show clear signs of convergence, suggesting that each technique has been sufficiently exercised and further gains would be minimal. Among them, TestTailor exhibits the steepest and most sustained growth, consistently outperforming TELPA and CoverUp in coverage improvement per unit of LLM usage. This underscores that TestTailor delivers not only higher cost-effectiveness but also superior scalability as the budget of LLM calls increases.

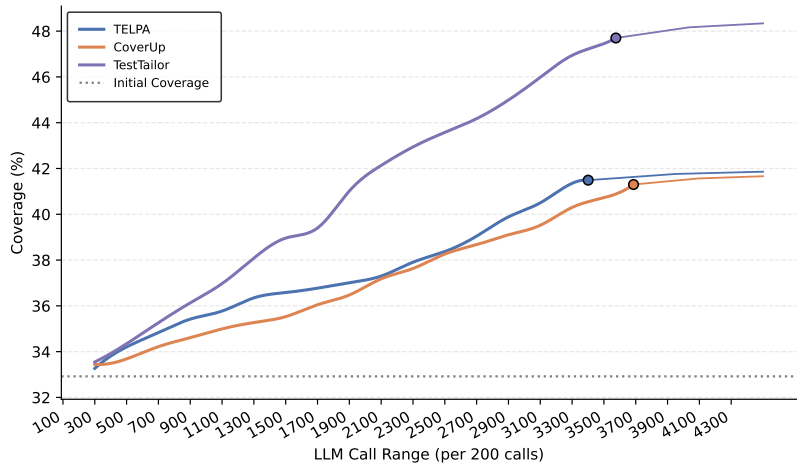

Fig. 9. The improvement curve of coverage versus the number of LLM API calls on the `ansible` package.

Conclusion 4: Cost-Effectiveness of TestTailor

TestTailor achieves the best balance between cost and effectiveness. It generates more coverage improvements per LLM call than TELPA and CoverUp, and reaches convergence at a higher coverage level with moderate resource consumption, demonstrating both efficiency and scalability.

[7] https://github.com/ansible/ansible/tree/devel

6 Discussion

Experimental Stability. Due to the high computational cost of LLM API calls, we executed the full benchmark of 486 modules once, relying on the large sample size to mitigate stochastic variance. To further verify stability, we repeated the experiments 10 times on three representative projects (`tqdm`, [8] `sanic`, [9] and `cookiecutter` [10]). The results showed minimal variance (maximum standard deviation of 3.98% in `tqdm`) and 100% ranking consistency, with TestTailor outperforming all baselines in all 30 runs. This confirms the robustness of the reported coverage gains.

Robustness with Strong Baselines. TestTailor builds on top of existing test suites by mining path-proximal information and leveraging coverage gaps. To examine its robustness under strong baselines, we conducted an additional experiment on 17 projects sampled from the CODAMOSA dataset. The initial test suites were generated using `claude-3-7-sonnet-20250219` [4], which already achieved average high coverage of 88.6% (statement) and 78.3% (branch). Applying TestTailor further increased coverage to 96.6% and 92.3%, respectively. This shows that TestTailor is not only effective in low-coverage settings but also capable of uncovering residual paths when the baseline is already strong—making it suitable for regression testing or CI/CD pipelines where projects typically maintain mature test suites. Moreover, although `claude-3-7-sonnet-20250219` ranks higher than the default backend on the SWE-bench benchmark [18], [11] the additional gains achieved here suggest that coverage improvements depend more on the guidance provided by TestTailor than on the raw code generation ability of the underlying LLM. This indicates that even comparatively weaker models can achieve substantial gains when properly constrained toward coverage-relevant goals.

Robustness on Developer-Written Test Suites. Beyond automatically generated baselines, we evaluated TestTailor on 24 modules sampled from CODAMOSA benchmark (`tqdm`, `sanic`, and `youtube-dl` [12]) using their original developer-written suites. TestTailor successfully boosted statement and branch coverage by 13.37 and 9.48 percentage points, respectively. This substantial improvement indicates that our tool can identify and cover complex corner cases that human developers overlook, demonstrating its practical value in enhancing mature, real-world test suites.

7 Threats to Validity

Assertion Generation. While TestTailor focuses primarily on generating test inputs to improve coverage, it does not explicitly address the generation of assertions. Similar to most existing work on LLM-based test generation (e.g., [3, 8, 22, 31, 36, 38, 39]), our current design retains only those cases that execute successfully and contribute additional coverage, while discarding those leading to runtime failures. This choice means that potential bug-revealing tests without proper assertions may be overlooked. Prior research on automatic assertion generation (e.g., [7, 12, 35]) has investigated ways to improve the quality and correctness of assertions. Integrating such techniques with TestTailor represents a promising direction for future work, enabling not only broader path coverage but also stronger fault-detection capabilities.

Challenging Scenarios. TestTailor may be less effective in two structured cases. First, in cases involving nested interprocedural constraints, where branch predicates depend on invoked methods (e.g., `if (methodA())`), TestTailor captures the return-value requirement of the call but does not explicitly recover the internal logic governing that return value (e.g., `methodA` internally requires `a > 0`). As a result, the LLM must independently infer such hidden state preconditions, which

[8] https://github.com/tqdm/tqdm
[9] https://github.com/sanic-org/sanic
[10] https://github.com/cookiecutter/cookiecutter
[11] This comparison result was obtained from https://www.swebench.com/index.html.
[12] https://github.com/ytdl-org/youtube-dl

increases generation uncertainty and may lead to incorrect or suboptimal test inputs. Second, when target branches require complex setups absent from path-proximal tests (e.g., network mocking or intricate object initialization), insufficient contextual signals may hinder valid test construction. Future improvements could integrate interprocedural call–sequence mining from static or dynamic analyses, and expand context retrieval to mine project-wide object construction patterns, thereby recovering implicit state constraints and realistic setup logic.

8 Related Work

CODAMOSA [21] integrates a code LLM (Codex) into search-based test generation, querying it for realistic test cases whenever evolutionary search stalls, and then mutating those examples to escape coverage plateaus. As such, CODAMOSA represents a hybrid LLM-and-search-based approach, in which the LLM serves as an auxiliary seed provider to the underlying SBST engine rather than as the primary driver of test generation.

ChatTester [40] first asks ChatGPT to infer the intention of the focal method and then prompts it again to generate a test driven by that intention. TELPA [39] improves LLM-based test generation for hard-to-cover branches by first, similar to ChatTester, asking the functionality of function, then enriching the prompt with program-analysis results (object-construction sequences, interprocedural dependencies) and counter-examples. Compared with TELPA, TestTailor offers finer-grained guidance by analyzing counter-examples' path behaviors to derive actionable constraints for reaching uncovered code.

CoverUp [3] identifies the code segments (e.g., functions or classes) that still lack coverage, and embeds this missing coverage information directly into the prompt. During generation, the LLM can iteratively request additional code context through tool functions, enabling it to produce more targeted and effective tests.

SymPrompt [31] statically derives approximate path constraints from the focal method's branches, reduces them into a minimal linear-basis set, and transforms each path into a dedicated prompt specifying the conditions to satisfy and the expected outcome. In a similar vein, PANTA [15] detects uncovered execution paths and directs the LLM to generate new tests that exercise those paths.

HITS [36] targets complex methods by decomposing them into slices and leveraging Chain-of-Thought [37] to guide step-by-step test generation, thereby achieving higher coverage.

There are also techniques that focus on the iterative refinement process. ChatUniTest [8] extends this idea by combining rule-based repair with LLM-based repair, effectively addressing syntax, compilation, and runtime errors. TestART [14] further leverages template-based repair and re-prompting strategies to fix compilation issues during test generation.

Beyond coverage-based methods, some approaches leverage mutation feedback to improve LLM-generated tests. For example, MuTAP [10], ACH [16], and PRIMG [6] use surviving or targeted mutants to guide the generation of more fault-revealing tests. These techniques focus on mutation scores and bug detection instead of code coverage.

The above approaches aim to improve test generation either by increasing coverage or by enhancing accuracy. Coverage-oriented methods vary in their guidance strategies: CODAMOSA injects new seeds to escape coverage plateaus; ChatTester extracts method intents, while TELPA emphasizes functionality and further leverages object-construction sequences and counterexamples; SymPrompt and PANTA design prompts around execution paths; CoverUp provides contextual information through tool functions; and HITS decomposes complex methods and applies chain-of-thought reasoning. While these techniques enrich the LLM with structural or contextual information, most still operate at a relatively coarse granularity, leaving the model to infer how to reach uncovered code. In contrast, TestTailor bridges this gap by leveraging path-proximal tests to capture the precise divergence point from the target path and extracting actionable constraints via symbolic

execution. This fine-grained, path-oriented guidance allows TestTailor to provide LLMs with not just *what* remains uncovered, but also *how* to reach it, thereby achieving higher coverage with fewer redundant tests.

9 Conclusion

This paper introduces TestTailor, a path-proximal test-guided approach to LLM-based test generation. By reusing path-proximal tests and extracting symbolic constraints, TestTailor provides fine-grained, unit-level guidance. Our results show that the reuse of existing tests combined with detailed path guidance is the primary driver of coverage improvement, and that TestTailor significantly outperforms state-of-the-art baselines in terms of coverage, accuracy, and cost-effectiveness across diverse open-source projects.

10 Data Availability

The replication package containing the source code of TestTailor, experimental scripts, and all data used in this study is publicly available at: https://github.com/Dortheay/TestTailor.

Acknowledgments

This work was supported by the National Key Research and Development Program of China under Grant No. 2024YFE0204200 and the National Natural Science Foundation of China under Grant No. 62372005.

References

[1] Aldeida Aleti, I. Moser, and Lars Grunske. 2017. Analysing the fitness landscape of search-based software testing problems. Automated Software Engg. 24, 3 (Sept. 2017), 603–621. doi:10.1007/s10515-016-0197-7

[2] Nadia Alshahwan, Jubin Chheda, Anastasia Finogenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Automated Unit Test Improvement using Large Language Models at Meta. In Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (Porto de Galinhas, Brazil) (FSE 2024). Association for Computing Machinery, New York, NY, USA, 185–196. doi:10.1145/3663529.3663839

[3] Juan Altmayer Pizzorno and Emery D. Berger. 2025. CoverUp: Effective High Coverage Test Generation for Python. Proceedings of the ACM on Software Engineering 2, FSE (June 2025), 2897–2919. doi:10.1145/3729398

[4] Anthropic. 2025. Claude Models Overview. https://docs.anthropic.com/en/docs/about-claude/models/overview. Accessed: 2025-09-11.

[5] Arianna Blasi, Alessandra Gorla, Michael D. Ernst, and Mauro Pezzè. 2023. Call Me Maybe: Using NLP to Automatically Generate Unit Test Cases Respecting Temporal Constraints. In Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (Rochester, MI, USA) (ASE '22). Association for Computing Machinery, New York, NY, USA, Article 19, 11 pages. doi:10.1145/3551349.3556961

[6] Mohamed Salah Bouafif, Mohammad Hamdaqa, and Edward Zulkoski. 2025. PRIMG : Efficient LLM-driven Test Generation Using Mutant Prioritization. In Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE '25). Association for Computing Machinery, New York, NY, USA, 1107–1116. doi:10.1145/3756681.3756991

[7] Junjie Chen, Yanwei Bai, Dan Hao, Lingming Zhang, Lu Zhang, Bing Xie, and Hong Mei. 2016. Supporting oracle construction via static analysis. In Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (Singapore, Singapore) (ASE '16). Association for Computing Machinery, New York, NY, USA, 178–189. doi:10.1145/2970276.2970366

[8] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. ChatUniTest: A Framework for LLM-Based Test Generation. In Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (Porto de Galinhas, Brazil) (FSE 2024). Association for Computing Machinery, New York, NY, USA, 572–576. doi:10.1145/3663529.3663801

[9] Ermira Daka and Gordon Fraser. 2014. A Survey on Unit Testing Practices and Problems. 2014 IEEE 25th International Symposium on Software Reliability Engineering (2014), 201–211. doi:10.1109/ISSRE.2014.11

[10] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. 2024. Effective test generation using pre-trained Large Language Models and mutation testing. Inf. Softw. Technol. 171, C (July 2024), 17 pages. doi:10.1016/j.infsof.2024.107468

[11] DeepSeek. 2025. News Article: news1226. https://api-docs.deepseek.com/news/news1226. Accessed: 2025-09-11.

[12] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K. Lahiri. 2022. TOGA: a neural method for test oracle generation. In Proceedings of the 44th International Conference on Software Engineering (Pittsburgh, Pennsylvania) (ICSE '22). Association for Computing Machinery, New York, NY, USA, 2130–2141. doi:10.1145/3510003.3510141

[13] Gordon Fraser and Andrea Arcuri. 2014. A Large-Scale Evaluation of Automated Unit Test Generation Using EvoSuite. ACM Trans. Softw. Eng. Methodol. 24, 2, Article 8 (Dec. 2014), 42 pages. doi:10.1145/2685612

[14] Siqi Gu, Chunrong Fang, Quanjun Zhang, Fangyuan Tian, and Zhenyu Chen. 2024. TestART: Improving LLM-based Unit Test via Co-evolution of Automated Generation and Repair Iteration. doi:10.48550/arXiv.2408.03095

[15] Sijia Gu, Noor Nashid, and Ali Mesbah. 2025. LLM Test Generation via Iterative Hybrid Program Analysis. (03 2025). doi:10.48550/arXiv.2503.13580

[16] Mark Harman, Jillian Ritchey, Inna Harper, Shubho Sengupta, Ke Mao, Abhishek Gulati, Christopher Foster, and Hervé Robert. 2025. Mutation-Guided LLM-based Test Generation at Meta. Association for Computing Machinery, New York, NY, USA, 180–191. doi:10.1145/3696630.3728544

[17] Chris Hathhorn. 2012. Engineering a compiler, second edition by Keith D. Cooper and Linda Torczon. SIGSOFT Softw. Eng. Notes 37, 1 (Jan. 2012), 36–37. doi:10.1145/2088883.2088908

[18] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770 [cs.CL] doi:10.48550/arXiv.2310.06770

[19] Claus Klammer and Albin Kern. 2015. Writing unit tests: It's now or never! (05 2015). doi:10.1109/ICSTW.2015.7107469

[20] Elson Kurian, Daniela Briola, Pietro Braione, and Giovanni Denaro. 2023. Automatically generating test cases for safety-critical software via symbolic execution. Journal of Systems and Software 199 (2023), 111629. doi:10.1016/j.jss.2023.111629

[21] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). 919–931. doi:10.1109/ICSE48619.2023.00085

[22] Runlin Liu, Zhe Zhang, Yunge Hu, Yuhang Lin, Xiang Gao, and Hailong Sun. 2025. LLM-based Unit Test Generation for Dynamically-Typed Programs. arXiv:2503.14000 [cs.SE] doi:10.48550/arXiv.2503.14000

[23] Stephan Lukasczyk and Gordon Fraser. 2022. Pynguin: automated unit test generation for Python. In Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings (Pittsburgh, Pennsylvania) (ICSE '22). Association for Computing Machinery, New York, NY, USA, 168–172. doi:10.1145/3510454.3516829

[24] Stephan Lukasczyk, Florian Kroiß, and Gordon Fraser. 2023. An empirical study of automated unit test generation for Python. Empirical Software Engineering 28, 2 (Jan 2023), 36. doi:10.1007/s10664-022-10248-w

[25] Microsoft. 2025. CODAMOSA Dataset. https://github.com/microsoft/codamosa-dataset. Accessed: 2025-09-11.

[26] Microsoft. 2025. CODAMOSA GitHub Repository. https://github.com/microsoft/codamosa. Accessed: 2025-09-11.

[27] Michael Olan. 2003. Unit testing: Test early, test often. Journal of Computing Sciences in Colleges - JCSC 19 (01 2003). doi:10.5555/948785.948830

[28] OpenAI. 2024. GPT-4o Model Documentation. https://platform.openai.com/docs/models/gpt-4o-2024-11-20. Accessed: 2025-09-11.

[29] Eunsik Park, Meehye Cho, and Chang-Seok Ki. 2009. Correct Use of Repeated Measures Analysis of Variance. The Korean journal of laboratory medicine 29 (03 2009), 1–9. doi:10.3343/kjlm.2009.29.1.1

[30] P. Runeson. 2006. A survey of unit testing practices. IEEE Software 23, 4 (2006), 22–29. doi:10.1109/MS.2006.91

[31] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-Aware Prompting: A Study of Coverage-Guided Test Generation in Regression Setting using LLM. Proc. ACM Softw. Eng. 1, FSE, Article 43 (July 2024), 21 pages. doi:10.1145/3643769

[32] Simone Scalabrino, Giovanni Grano, Dario Di Nucci, Michele Guerra, Andrea De Lucia, Harald C. Gall, and Rocco Oliveto. 2018. OCELOT: a search-based test-data generation tool for C. In Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (Montpellier, France) (ASE '18). Association for Computing Machinery, New York, NY, USA, 868–871. doi:10.1145/3238147.3240477

[33] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. IEEE Trans. Softw. Eng. 50, 1 (Jan. 2024), 85–105. doi:10.1109/TSE.2023.3334955

[34] Allen Talikan, Rufaida Salapuddin, Jainalyn Aksan, Adzhar Ismael, Rumida Jimlah, Nasra Idris, Rubina Dammang, Dermayna Jamar, Emelyn Sarahadil, and Rozia Ajan. 2025. ON PAIRED SAMPLES T-TEST: APPLICATIONS, EXAMPLES AND LIMITATIONS. (03 2025). doi:10.5281/zenodo.10987546

[35] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, and Neel Sundaresan. 2022. Generating accurate assert statements for unit test cases using pretrained transformers. In Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test (Pittsburgh, Pennsylvania) (AST '22). Association for Computing Machinery, New York, NY, USA, 54–64. doi:10.1145/3524481.3527220

[36] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. HITS: High-coverage LLM-based Unit Test Generation via Method Slicing. In Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (Sacramento, CA, USA) (ASE '24). Association for Computing Machinery, New York, NY, USA, 1258–1268. doi:10.1145/3691620.3695501

[37] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In Proceedings of the 36th International Conference on Neural Information Processing Systems (New Orleans, LA, USA) (NIPS '22). Curran Associates Inc., Red Hook, NY, USA, Article 1800, 14 pages. doi:10.5555/3600270.3602070

[38] Zhiyi Xue, Liangguo Li, Senyue Tian, Xiaohong Chen, Pingping Li, Liangyu Chen, Tingting Jiang, and Min Zhang. 2024. LLM4Fin: Fully Automating LLM-Powered Test Case Generation for FinTech Software Acceptance Testing. In Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (Vienna, Austria) (ISSTA 2024). Association for Computing Machinery, New York, NY, USA, 1643–1655. doi:10.1145/3650212.3680388

[39] Chen Yang, Junjie Chen, Bin Lin, Ziqi Wang, and Jianyi Zhou. 2025. Advancing Code Coverage: Incorporating Program Analysis with Large Language Models. ACM Trans. Softw. Eng. Methodol. (July 2025). doi:10.1145/3748505 Just Accepted.

[40] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and Improving ChatGPT for Unit Test Generation. Proc. ACM Softw. Eng. 1, FSE, Article 76 (July 2024), 24 pages. doi:10.1145/3660783

[41] Hong Zhu, Patrick A. V. Hall, and John H. R. May. 1997. Software unit test coverage and adequacy. ACM Comput. Surv. 29, 4 (Dec. 1997), 366–427. doi:10.1145/267580.267590

Received 2025-09-10; accepted 2025-12-22